WEEKLY ASSESSMENT - 4

1. Explain the break, continue, and pass statements with examples.

- **break:** Terminates the current loop execution prematurely and resumes control at the next statement immediately following the loop block.

```
for i in range(1, 6):
    if i == 3:
        break
    print(i)
# Output: 1, 2
```

- **continue:** Skips the remaining code inside the current loop iteration and immediately jumps back to the top of the loop for the next cycle.

```
for i in range(1, 5):
    if i == 3:
        continue
    print(i)
# Output: 1, 2, 4
```

- **pass:** A null statement that serves as a syntax placeholder. It executes nothing but is utilized where syntactical block definition is required but no action is yet planned.

```
def empty_function():
    pass  # Placeholder for future implementation
```

2. Convert a list of temperatures in Celsius to Fahrenheit using map().

Using the conversion formula *F = (C * 9/5) + 32*, we apply a lambda function inside Python's built-in map() function to process the entire sequence:

```
celsius_temps = [0, 20, 37, 100]

# Mapping the formula over the list
fahrenheit_temps = list(map(lambda c: (c * 9/5) + 32, celsius_temps))

print(fahrenheit_temps)
# Output: [32.0, 68.0, 98.6, 212.0]
```

3. Find the second smallest element in a list.

To safely find the second smallest value while ignoring exact duplicates, we convert the collection into a unique set, sort it, and then isolate the second index element:

```
numbers = [12, 5, 8, 5, 1, 9, 1]
```

```
# Remove duplicates using set() and sort the unique numbers
unique_nums = sorted(list(set(numbers)))

if len(unique_nums) >= 2:
    second_smallest = unique_nums[1]
    print("Second smallest element:", second_smallest)  # Output: 5
else:
    print("List does not contain a valid second smallest unique
element.")
```

4. Write a program to check if a year is a leap year.

A year is a leap year if it is divisible by 4, except for end-of-century years which must also be divisible by 400:

```
year = 2024

if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
    print(f"{year} is a leap year.")
else:
    print(f"{year} is not a leap year.")
```

5. How do you reverse a dictionary (keys become values and vice versa)?

A dictionary can be reversed cleanly using a standard dictionary comprehension mapping values back into keys:

```
original_dict = {'a': 1, 'b': 2, 'c': 3}

# Reversing keys and values
reversed_dict = {value: key for key, value in original_dict.items()}

print(reversed_dict)  # Output: {1: 'a', 2: 'b', 3: 'c'}
```

Note: This inversion strategy assumes that all values within the original dictionary are unique and hashable.

6. Demonstrate the use of any() and all() on a list of booleans.

- **any():** Evaluates to True if at least one single element inside the iterable is true.
- **all():** Evaluates to True only when every element inside the iterable is true.

```
bool_list1 = [True, False, True]
bool_list2 = [True, True, True]

print(any(bool_list1))  # Output: True (contains True elements)
print(all(bool_list1))  # Output: False (contains a False element)
print(all(bool_list2))  # Output: True (all values are True)
```

7. Write a script to flatten a nested list (e.g., [[1,2],[3,4]] to [1,2,3,4]).

A double-iteration list comprehension allows flattening a matrix-like sequence cleanly:

```
nested_list = [[1, 2], [3, 4]]

# Flattening via a nested list comprehension loop
flattened_list = [item for sublist in nested_list for item in
sublist]

print(flattened_list)  # Output: [1, 2, 3, 4]
```

8. Explain the difference between append() and extend() in lists.

Method	Behavior Description	List Length Mutation
append()	Appends its single argument object directly to the end of the target list as one atomic element.	Always grows list length by exactly +1.
extend()	Iterates through the provided collection argument, unpacking and joining every item individually onto the sequence.	Grows list length by +n elements (where n is the length of the passed collection).

```
# append() demonstration
list_a = [1, 2]
list_a.append([3, 4])
print(list_a)  # Output: [1, 2, [3, 4]]

# extend() demonstration
list_b = [1, 2]
list_b.extend([3, 4])
print(list_b)  # Output: [1, 2, 3, 4]
```

9. Write a script to find the "Longest Word" in a sentence.

By extracting whitespace-separated tokens with .split(), we can feed the word list to the max() built-in with a custom length measurement key:

```
sentence = "Python is an amazing and powerful programming language"

words = sentence.split()
longest_word = max(words, key=len)

print("Longest word:", longest_word)  # Output: programming
```

10. Use the collections.Counter module to find the 3 most common items in a list.

The standard library provides a specialized dictionary wrapper class Counter which includes a native .most_common() tracking routine:

```
from collections import Counter

items = ['apple', 'banana', 'apple', 'orange', 'banana', 'apple',
'grape', 'banana', 'banana']

# Count occurrences of elements
item_counts = Counter(items)

# Retrieve the top 3 high-frequency elements
top_three = item_counts.most_common(3)

print(top_three)
# Output: [('banana', 4), ('apple', 3), ('orange', 1)]
```